

UNIVERSIDAD DEL VALLE DE GUATEMALA

CC3067 - Redes

Sección 20

Ing. Jorge Yass



Uso de un protocolo existente

Nicolle Alexandra Gordillo, 22246

Guatemala, 8 de septiembre del 2026

1. Descripción general

Se implementó un chatbot anfitrión que se comunica con múltiples servidores MCP (Model Context Protocol) mediante JSON-RPC 2.0, construido íntegramente sobre la biblioteca estándar de Python y sin utilizar ningún SDK del protocolo. El caso de uso de industria corresponde a la gestión de inventario y pedidos de una distribuidora mayorista de productos de consumo.

El sistema consta de cuatro componentes:

Componente	Archivo	Rol en el protocolo
Chatbot	chatbot.py	Anfitrión: coordina los clientes y el LLM
Cliente MCP	mcp_client.py	Cliente: mantiene la conexión con cada servidor
Servidor local	server.py	Servidor sobre transporte stdio
Servidor remoto	server_http.py	Servidor sobre transporte HTTP (Cloud Run)

La lógica de negocio reside en tools.py y es compartida sin modificación por ambos servidores. La conexión con el LLM se aisló en llm_providers.py, lo que permite intercambiar el proveedor sin alterar el resto del sistema.

2. Especificación de los servidores MCP desarrollados

2.1 Identidad y capacidades

Atributo	Valor
Nombre	distribuidora-inventario
Versión	1.0.0
Versiones de protocolo soportadas	2025-11-25, 2025-06-18, 2025-03-26, 2024-11-05
Capacidades declaradas	tools (con listChanged: false)
Transporte local	stdio, mensajes delimitados por salto de línea
Transporte remoto	HTTP (Streamable HTTP)

El servidor declara únicamente la capacidad tools. En consecuencia, métodos como resources/list son rechazados con el código -32601, comportamiento coherente con lo anunciado durante el handshake.

2.2 Métodos del protocolo implementados

Método	Tipo	Descripción
initialize	Petición	Handshake; negocia versión e intercambia capacidades
notifications/initialized	Notificación	El cliente confirma que está listo; no genera respuesta
tools/list	Petición	Devuelve el catálogo con el JSON Schema de cada herramienta
tools/call	Petición	Invoca una herramienta por nombre con sus argumentos
ping	Petición	Verificación de conexión activa
shutdown	Petición	Cierre ordenado (extensión propia)

2.3 Herramientas expuestas

Los precios se expresan en quetzales (GTQ). Las bodegas son: 1 = Central, 2 = Occidente, 3 = Oriente.

buscar_productos

Busca en el catálogo por coincidencia parcial de nombre y/o categoría.

Parámetro	Tipo	Requerido	Descripción
consulta	string	No	Texto a buscar dentro del nombre del producto
categoria	string	No	Categoría exacta (8 valores admitidos)
limite	integer	No	Máximo de resultados, 1–50. Por defecto 10

consultar_stock

Devuelve las existencias de un producto.

Parámetro	Tipo	Requerido	Descripción
sku	string	Sí	Código del producto, formato XX-NNNN
bodega_id	integer	No	Restringe la consulta a una bodega

crear_pedido

Registra un pedido nuevo. Es la única herramienta que escribe.

Parámetro	Tipo	Requerido	Descripción
cliente_id	integer	Sí	Identificador del cliente
lineas	array	Sí	Objetos {sku, cantidad}, mínimo uno
bodega_id	integer	No	Bodega de despacho. Por defecto 1

La operación es atómica: se validan el cliente, la bodega y todas las líneas antes de escribir; si alguna falla, se revierte la transacción completa y no queda registro parcial ni descuento parcial de inventario.

consultar_pedido

Estado y detalle de un pedido.

Parámetro	Tipo	Requerido	Descripción
numero	string	Sí	Número de pedido, formato PED-AAAA-NNNN

historial_cliente

Pedidos recientes y monto acumulado de un cliente.

Parámetro	Tipo	Requerido	Descripción
cliente_id	integer	Sí	Identificador del cliente
limite	integer	No	Pedidos a listar, 1–50. Por defecto 5

convertir_total_a_dolares

Convierte el total de un pedido a dólares usando el tipo de cambio de referencia del Banco de Guatemala. Es la única herramienta que consume una fuente de datos externa.

Parámetro	Tipo	Requerido	Descripción
numero	string	Sí	Número del pedido a convertir

Si el servicio externo no responde, se utiliza una tasa de respaldo y el resultado incluye `consulta_en_linea: false`, de modo que el chatbot pueda advertir que el dato no proviene de la consulta en línea.

2.4 Endpoints del servidor remoto

Endpoint	Método	Descripción
/mcp	POST	Recibe un mensaje JSON-RPC y devuelve la respuesta
/mcp	DELETE	Cierra la sesión identificada por Mcp-Session-Id
/health	GET	Verificación de salud para el balanceador de la nube
/	GET	Información del servidor y nombres de las herramientas

2.5 Manejo de errores en dos niveles

La implementación distingue entre fallas del protocolo y fallas del negocio, distinción que resultó central en el diseño.

Errores de protocolo — se devuelven como objeto error de JSON-RPC:

Código	Condición
-32700	JSON malformado en la entrada
-32600	Campo jsonrpc o method ausente o inválido
-32601	Método no soportado
-32602	Parámetros inválidos o herramienta inexistente
-32603	Error interno no controlado

Errores de negocio: se devuelven como respuesta exitosa cuyo resultado incluye `isError: true` y un mensaje legible. Casos como existencias insuficientes o SKU inexistente caen aquí: la llamada fue sintácticamente válida, de modo que el modelo debe leer el mensaje y reaccionar (proponer otra bodega, solicitar un código válido) en lugar de interpretarlo como una falla de transporte.

2.6 Fuentes de datos

Base de datos SQLite con seis tablas (productos, bodegas, existencias, clientes, pedidos, pedido_lineas). Al no disponerse de información real de una empresa, se pobló con un conjunto sintético generado por `db/seed.py` con semilla fija (3067): 92 productos en 8 categorías, 3 bodegas, 20 clientes y 40 pedidos históricos con 117 líneas de detalle. La semilla fija garantiza que el conjunto sea idéntico en cualquier máquina y que el escenario de demostración sea reproducible.

La única fuente externa es el servicio web del Banco de Guatemala (TipoCambioDia), consumido por SOAP sobre HTTP.

3. Análisis de la comunicación con Wireshark

3.1 Condiciones de la captura

Parámetro	Valor
Archivo	capturas/local_http.pcapng
Interfaz	Adapter for loopback traffic capture
Filtro de captura	tcp port 8080
Paquetes capturados	246
Duración	186.95 s
Cliente	Python-urllib/3.10 (mcp_client.py)

Parámetro	Valor
Servidor	distribuidora-inventario/1.0.0 Python/3.10.5

La captura se realizó contra el servidor ejecutándose localmente sobre HTTP en claro. Esta decisión fue deliberada: Cloud Run sirve exclusivamente sobre HTTPS, y una captura del servicio remoto muestra únicamente registros Application Data cifrados por TLS, en los que no es posible leer un solo mensaje JSON-RPC. Dado que el objetivo del análisis es clasificar los mensajes del protocolo, se capturó el mismo intercambio sobre transporte no cifrado. La sección 3.7 comenta las diferencias que introduce el despliegue remoto.

La captura contiene dos sesiones MCP. La primera (tramas 11–98) corresponde a una prueba inicial de conexión; la segunda (tramas 109–246) contiene el escenario completo y es la que se analiza a continuación.

3.2 Clasificación de los mensajes JSON-RPC

El enunciado solicita distinguir tres categorías. La tabla siguiente empareja cada mensaje de la sesión completa con su respuesta mediante el campo id, que es el mecanismo de correlación de JSON-RPC.

Trama	Tipo	id	Método / contenido	Respuesta
112, 114	Sincronización	1	initialize	116, 118
137, 139	Notificación	—	notifications/initialized	141 (202)
160, 162	Petición	2	tools/list	164, 166
185, 187	Petición	3	tools/call → buscar_productos	189, 191
210, 212	Petición	4	tools/call → consultar_stock	214, 216
235, 237	Petición	5	shutdown	239, 241

Cada mensaje ocupa dos tramas porque el cliente envía las cabeceras HTTP y el cuerpo en segmentos TCP separados; lo mismo ocurre en las respuestas del servidor.

Mensajes de sincronización

Existen en dos niveles que conviene no confundir. En el nivel de transporte, el three-way handshake TCP que precede a cada petición. En el nivel de aplicación, el intercambio initialize/respuesta, donde se negocia la versión del protocolo y se anuncian capacidades:

```
→ {"jsonrpc": "2.0", "id": 1, "method": "initialize",
    "params": {"protocolVersion": "2025-06-18", "capabilities": {},
               "clientInfo": {"name": "chatbot-distribuidora",
                              "version": "1.0.0"}}}
```

```
← {"jsonrpc": "2.0", "id": 1, "result": {"protocolVersion": "2025-06-18",  
  "capabilities": {"tools": {"listChanged": false}},  
  "serverInfo": {"name": "distribuidora-inventario",  
    "version": "1.0.0"}, ...}}
```

El cliente solicitó la versión 2025-06-18 y el servidor la aceptó por estar entre las que soporta. En la respuesta (trama 116) el servidor asignó además el identificador de sesión mediante la cabecera `Mcp-Session-Id: 4c2b276f84334d30ba1da6e360ad895e`, que el cliente reenvió en todas las peticiones posteriores (verificable en las cabeceras de las tramas 185 y 210).

Notificaciones

La trama 139 transporta un mensaje con `method` pero sin `id`:

```
→ {"jsonrpc": "2.0", "method": "notifications/initialized"}
```

Al carecer de `id`, no espera respuesta. En la captura esto se distingue con claridad porque el servidor contesta HTTP/1.1 202 Accepted con `Content-Length: 0` (trama 141), en lugar del 200 OK con cuerpo que devuelve a las peticiones. La diferencia entre petición y notificación, que en la especificación de JSON-RPC es la presencia del campo `id`, se manifiesta aquí como una diferencia de código de estado HTTP.

Peticiones

Llevan `method` e `id` y esperan respuesta. La trama 187 contiene:

```
→ {"jsonrpc": "2.0", "id": 3, "method": "tools/call",  
  "params": {"name": "buscar_productos",  
    "arguments": {"consulta": "aceite", "limite": 1}}}
```

Respuestas

Llevan el mismo `id` de la petición que las originó y el campo `result` o `error`, nunca ambos. La trama 191 responde a la anterior:

```
← {"jsonrpc": "2.0", "id": 3, "result": {"content": [{"type": "text",  
  "text": "{ \"encontrados\": 1, \"productos\": [{  
    \"sku\": \"AB-0009\", ... }] }"}],  
  "isError": false}}
```

El resultado de una herramienta viaja como una lista de bloques de contenido; el objeto JSON producido por la lógica de negocio se serializa dentro de un bloque de texto.

3.3 Capa de enlace

La captura se realizó sobre la interfaz de loopback, por lo que no existen tramas Ethernet ni direcciones MAC. El encabezado de enlace es el pseudo-encabezado Null/Loopback de 4 bytes que Windows antepone al tráfico interno; en la captura se observa como 02 00 00 00, donde el valor 2 identifica a la familia de protocolos IPv4.

Esta ausencia es en sí misma un resultado del análisis: la comunicación entre dos procesos de la misma máquina no atraviesa ningún medio físico compartido, de modo que el direccionamiento de nivel 2 —cuya función es identificar estaciones dentro de un segmento— carece de sentido. El sistema operativo entrega los paquetes directamente a través de la pila de red sin involucrar controlador ni adaptador alguno.

Como consecuencia, tampoco se observa segmentación por MTU. La MTU de la interfaz de loopback en Windows es de 65 535 bytes, muy superior a los 1 500 de Ethernet. La respuesta más grande de la sesión, el catálogo completo de herramientas devuelto por `tools/list` (trama 166), mide 3 949 bytes y viaja en un único segmento TCP. Sobre una red Ethernet real ese mismo mensaje se habría fragmentado en al menos tres segmentos y Wireshark lo habría señalado como TCP segment of a reassembled PDU.

3.4 Capa de red

Todo el tráfico ocurre entre 127.0.0.1 y 127.0.0.1: origen y destino son la misma dirección de loopback. Los paquetes son IPv4 con un encabezado de 20 bytes sin opciones, y no se observa fragmentación IP en ninguna trama.

Al no salir del equipo, los paquetes no atraviesan enrutadores y el campo TTL no se decrementa. Esto contrasta con el servidor desplegado en Cloud Run, donde el destino no es siquiera el contenedor sino un balanceador de carga de Google: una consulta DNS al nombre del servicio devuelve varias direcciones IP distintas, evidencia de que el servicio se encuentra detrás de infraestructura compartida y de que la dirección de destino visible no identifica a la máquina que ejecuta el código.

3.5 Capa de transporte

El puerto de destino es siempre 8080. El puerto de origen es efímero y cambia en cada petición, lo cual constituye el hallazgo más relevante de esta capa. Los puertos observados fueron 54883, 54886, 54888, 51993, 51995, 51997, 51999, 52001, 52003 y 52005: diez puertos distintos para diez mensajes.

La causa se encuentra en las cabeceras HTTP de las peticiones, que incluyen `Connection: close`. El cliente `urllib` no reutiliza conexiones, de modo que cada mensaje JSON-RPC abre y cierra su propia conexión TCP completa. El ciclo de vida de una de ellas, tomado de la petición `tools/call` → `buscar_productos`, ilustra el patrón:

Trama	Flags	Descripción
182	SYN	El cliente inicia la conexión, seq = 3706423179
183	SYN, ACK	El servidor responde, seq = 3157300578, ack = 3706423180
184	ACK	El cliente confirma; conexión establecida

Trama	Flags	Descripción
185	PSH, ACK	Cabeceras HTTP (270 bytes)
186	ACK	El servidor acusa recibo
187	PSH, ACK	Cuerpo JSON-RPC (141 bytes)
188	ACK	El servidor acusa recibo
189	PSH, ACK	Cabeceras de respuesta (178 bytes)
191	PSH, ACK	Cuerpo de la respuesta (453 bytes)
193	FIN, ACK	El servidor inicia el cierre
194	ACK	El cliente confirma
195	FIN, ACK	El cliente cierra su sentido
196	ACK	El servidor confirma; conexión terminada

Se observan los tres mecanismos característicos de TCP. El establecimiento mediante three-way handshake con números de secuencia iniciales aleatorios, y el cierre de cuatro vías donde cada extremo termina su sentido de forma independiente. Los números de secuencia avanzan exactamente en la cantidad de bytes transmitidos: tras enviar 270 bytes de cabeceras, el seq del cliente pasa de 3706423180 a 3706423450, y el servidor confirma con ese mismo ack. La ventana de recepción se anuncia inicialmente en 65 535 bytes y desciende ligeramente conforme se transfieren datos (255 → 254 → 253), reflejando el mecanismo de control de flujo.

Nótese que en esta conexión el intercambio útil ocupa cuatro tramas con datos, mientras que el establecimiento y el cierre consumen siete. Abrir una conexión por mensaje impone un costo notable: sobre una red real, cada handshake añadiría al menos un tiempo de ida y vuelta antes de poder transmitir. Reutilizar la conexión mediante keep-alive sería la optimización natural, y es una mejora concreta identificada por este análisis.

Los tiempos de respuesta del servidor son consistentemente bajos. Entre la recepción del cuerpo (trama 187) y el envío de la respuesta (trama 189) transcurren 66 ms en buscar_productos y 5.6 ms en consultar_stock; la diferencia se explica por el LIKE sobre el nombre del producto frente a la búsqueda por clave primaria.

3.6 Capa de aplicación

En esta capa se apilan tres protocolos, y conviene distinguirlos.

HTTP actúa como transporte. Todas las peticiones son POST /mcp con Content-Type: application/json y Content-Length explícito. Las cabeceras completas de la trama 185 son:

```
POST /mcp HTTP/1.1
Accept-Encoding: identity
```

```
Content-Length: 141
Host: localhost:8080
User-Agent: Python-urllib/3.10
Content-Type: application/json
Accept: application/json, text/event-stream
Mcp-Session-Id: 4c2b276f84334d30ba1da6e360ad895e
Connection: close
```

La cabecera Accept anuncia que el cliente admite tanto application/json como text/event-stream, conforme al transporte Streamable HTTP de MCP, que permite al servidor responder con un flujo de eventos cuando la operación lo requiere. En esta implementación todas las respuestas fueron application/json.

Los códigos de estado observados fueron 200 OK para las respuestas a peticiones y 202 Accepted para la notificación.

JSON-RPC 2.0 define el formato de los mensajes: los campos jsonrpc, method, params, id, result y error. Es el nivel donde ocurre la correlación entre peticiones y respuestas.

MCP aporta la semántica: qué métodos existen, qué significan y en qué orden deben invocarse. El protocolo exige que initialize preceda a cualquier otra operación, y que el cliente confirme con notifications/initialized antes de utilizar las herramientas.

3.7 Comparación entre el servidor local y el remoto

El contraste entre ambos transportes es, a juicio de este análisis, la observación más significativa del proyecto.

El servidor local se comunica por stdio: el anfitrión lo lanza como proceso hijo y conversa con él a través de tuberías del sistema operativo. En ese caso no existen capas de red ni de transporte en absoluto. No hay direcciones IP, ni puertos, ni números de secuencia, ni control de flujo; el sistema operativo transfiere bytes entre dos procesos y no hay nada que capturar con Wireshark. La conexión no es una dirección sino la relación padre-hijo entre procesos.

El servidor remoto atraviesa la pila completa: enlace, red, transporte y aplicación, con el añadido de TLS entre transporte y aplicación.

Sin embargo, los mensajes JSON-RPC intercambiados son idénticos en ambos casos. El mismo initialize, la misma notificación, los mismos tools/call con los mismos argumentos y los mismos resultados. Esto se verificó en la implementación: la clase MCPClient es la misma para ambos y únicamente cambia el objeto de transporte que recibe; del lado del servidor, server_http.py importa process_message de server.py y reutiliza tools.py sin modificación alguna.

Esta es la separación de capas del modelo OSI observada de forma directa: un protocolo de capa de aplicación es independiente de las capas que lo transportan. La misma conversación puede viajar por tuberías de un sistema operativo o por una conexión TLS a través de Internet, y ni el cliente ni el servidor necesitan enterarse.

4. Dificultades encontradas

Créditos de API no disponibles. El enunciado indica que Anthropic proporciona 5 dólares de crédito gratuito sin requerir tarjeta. Al crear la cuenta, el saldo resultó ser de 0.00 US\$ y la API respondió con HTTP 400: Your credit balance is too low. Dado que el enunciado sugiere Anthropic sin exigirlo, se optó por migrar a Gemini, que ofrece capa gratuita.

Diseño de la migración. En lugar de reemplazar un proveedor por otro, se extrajo la conexión con el LLM a un módulo independiente (llm_providers.py) donde el anfitrión mantiene el historial en un formato neutral y cada proveedor lo traduce a su propia API. Cambiar de LLM pasó a ser cuestión de una bandera en la línea de comandos, y el cliente MCP, los servidores y la lógica del anfitrión quedaron intactos.

Incompatibilidad de esquemas con Gemini. La API rechazó las peticiones con HTTP 400: Invalid value at ... enum[0] (TYPE_STRING). Gemini acepta únicamente un subconjunto de JSON Schema y, en particular, sólo admite enum sobre campos de tipo cadena. Los esquemas del servidor MCP declaraban bodega_id como entero con enum: [1, 2, 3], además de emplear additionalProperties, default, minimum, maximum y minItems, ninguno de los cuales es reconocido. Se implementó la función limpiar_esquema(), que recorre el esquema, retira las claves no soportadas y traslada los valores de los enums numéricos a la descripción textual, de modo que el modelo conserve la información aunque ya no esté validada por el esquema.

Firmas de pensamiento. Los modelos Gemini 3.x acompañan cada functionCall con un campo thoughtSignature y exigen recibirlo intacto al reenviar el historial; de lo contrario responden HTTP 400. La implementación original reconstruía las partes del mensaje a partir del formato neutral, con lo que la firma se perdía. La solución fue conservar la respuesta original del proveedor y reenviarla verbatim, en lugar de rearmarla. Este enfoque resulta además más robusto ante campos obligatorios que puedan añadirse en el futuro.

Límites de cuota. La capa gratuita de Gemini impone un límite de peticiones que se alcanzó durante las pruebas (HTTP 429: RESOURCE_EXHAUSTED). Se implementó un mecanismo de reintento que distingue los códigos transitorios (429, 500, 502, 503, 504) de los errores de la petición (400, 401, 404), que no tiene sentido reintentar. El tiempo de espera se extrae del propio mensaje de error, que indica cuántos segundos deben transcurrir, contemplando tanto segundos como milisegundos.

Permisos en Google Cloud. El primer despliegue falló con PERMISSION_DENIED: Build failed because the default service account is missing required IAM permissions. En proyectos creados recientemente, la cuenta de servicio de Compute no recibe automáticamente los permisos que Cloud Build necesita para leer el código fuente subido a Cloud Storage. Se resolvió asignando explícitamente los roles cloudbuild.builds.builder, storage.objectViewer, artifactregistry.writer y logging.logWriter.

Servidores MCP oficiales. Los servidores Filesystem y Git no conectaban inicialmente: el primero terminaba sin responder al initialize y el segundo fallaba con WinError 2. Las causas fueron que el directorio de trabajo no existía y que uv no estaba instalado. La instalación de uv requirió además ajustar la política de ejecución de PowerShell.

Captura sobre HTTPS. Cloud Run sirve exclusivamente sobre TLS, por lo que una captura directa del servicio remoto muestra únicamente registros Application Data cifrados. Se optó por capturar el mismo intercambio contra el servidor local sobre HTTP en claro, dado que el objetivo del análisis es clasificar los mensajes del protocolo y éstos son idénticos en ambos transportes.

5. Conclusiones

La separación entre protocolo y lógica de negocio determinó la viabilidad del proyecto. La decisión de mantener el ruteo de mensajes JSON-RPC en server.py y la lógica en tools.py se tomó al construir el servidor local, antes de saber cómo se implementaría el remoto. Esa decisión permitió que server_http.py importara la capa de protocolo y reutilizara la de negocio sin modificar una sola línea. La simetría se repitió del lado del cliente: MCPClient es idéntico para stdio y HTTP, y sólo cambia el objeto de transporte que se le inyecta. Si ambas responsabilidades se hubieran mezclado, la segunda parte del proyecto habría exigido duplicar código o refactorizar a mitad del desarrollo.

MCP resuelve la interoperabilidad del lado del servidor, pero no del lado del modelo. El protocolo estandariza cómo un servidor declara sus herramientas: un mismo servidor MCP funciona con cualquier anfitrión sin adaptaciones. Sin embargo, cada API de LLM consume esas declaraciones con reglas propias. Trasladar el catálogo de herramientas de Anthropic a Gemini exigió reescribir la estructura de los mensajes, limpiar los esquemas y preservar campos específicos del proveedor. La conclusión es que MCP desplaza el problema de interoperabilidad, no lo elimina: la traducción entre el catálogo MCP y el formato de cada modelo recae sobre el anfitrión, que es exactamente el componente que el protocolo no estandariza.

Distinguir errores de protocolo de errores de negocio resultó ser una decisión de diseño con consecuencias prácticas. Un mensaje mal formado y una operación que no procede son situaciones distintas: la primera indica que el cliente está mal implementado, la segunda que el estado del sistema no permite lo solicitado. Devolver un error JSON-RPC en el segundo caso habría hecho que el modelo interpretara un stock insuficiente como una falla de comunicación. Al devolverlo como resultado válido con isError: true y un mensaje redactado en lenguaje natural, el modelo lo lee y propone una alternativa, y la conversación continúa.

El análisis con Wireshark hizo visible la independencia entre capas. Comparar ambos transportes mostró de forma concreta lo que el modelo OSI plantea de forma abstracta. La misma conversación JSON-RPC (los mismos métodos, los mismos identificadores, los mismos resultados) viaja en un caso por tuberías del sistema operativo, sin tocar la pila TCP/IP, y en el

otro atraviesa enlace, red, transporte y TLS hasta un balanceador de carga en otro continente. Ninguno de los dos extremos necesita saber cuál de las dos situaciones ocurre.

La captura reveló una ineficiencia que el código no hacía evidente. Los diez puertos efímeros distintos mostraron que cada mensaje JSON-RPC abría y cerraba su propia conexión TCP, consecuencia de la cabecera Connection: close que envía urllib por defecto. En la conexión analizada, siete de las trece tramas se dedicaron a establecer y cerrar la conexión, y sólo cuatro a transportar datos. Sobre loopback el costo es imperceptible, pero contra un servicio en la nube cada mensaje pagaría un handshake TCP y una negociación TLS completos. Es un problema que no se manifiesta leyendo el código (la llamada a urlopen parece igual en ambos casos) y que sólo aparece al observar el tráfico.

Implementar el protocolo manualmente tuvo valor pedagógico. Construir los mensajes JSON-RPC a mano obligó a comprender por qué una notificación carece de id, por qué el id debe coincidir entre petición y respuesta, y qué se negocia realmente durante el handshake. Con un SDK esas decisiones habrían quedado ocultas tras una llamada a función, y la clasificación de mensajes en la captura habría sido un ejercicio de reconocimiento de patrones en lugar de la verificación de algo previamente comprendido.

6. Comentario sobre el proyecto

El proyecto resultó valioso por combinar tres ámbitos que suelen estudiarse por separado: el diseño de un protocolo de capa de aplicación, su despliegue en infraestructura real y el análisis del tráfico que genera. Trabajar con un protocolo reciente añadió una dificultad no prevista pero instructiva: varias de las incidencias documentadas en la sección 4 se originaron en cambios de las APIs ocurridos después de que se redactara el enunciado (los créditos gratuitos que dejaron de existir, el identificador de modelo retirado, el campo thoughtSignature introducido en una versión nueva). Diagnosticarlas a partir de los mensajes de error, en ausencia de documentación previa, fue probablemente el aprendizaje más transferible del proyecto.

La restricción de implementar JSON-RPC manualmente, que inicialmente parecía un obstáculo, terminó siendo el aspecto más formativo del trabajo.

7. Referencias

- Especificación de JSON-RPC 2.0 — <https://www.jsonrpc.org/>
- Arquitectura de MCP — <https://modelcontextprotocol.io/docs/learn/architecture>
- Especificación de MCP — <https://modelcontextprotocol.io/specification/2025-11-25>
- Servidores MCP oficiales — <https://github.com/modelcontextprotocol/servers>
- Documentación de la API de Gemini — <https://ai.google.dev/gemini-api/docs>

- Despliegue de servidores MCP en Cloud Run —
<https://cloud.google.com/blog/topics/developers-practitioners/build-and-deploy-a-remote-mcp-server-to-google-cloud-run-in-under-10-minutes>